ИТОГОВАЯ РАБОТА

Программа для подсчета статистики текстового файла

Кейс 5

Выполнил: Хакимов Ильдар Фаридович
Программа обучения: Python-разработчик с использованием инструментов ИИ
Группа: ПР-5-Р
Руководитель: Гирич Полина Ивановна

Санкт-Петербург, 2026

2 Описание решения кейса

Выбранный кейс и постановка задачи

Выбран кейс 5 «Программа для подсчёта статистики текстового файла». По заданию требуется создать программу на Python, которая читает текстовый файл, рассчитывает базовые показатели, находит часто встречающиеся слова, позволяет выбрать файл и вид статистики, а также выводит и сохраняет результат.

Цель и задачи решения

Цель проекта — сделать простое настольное приложение, с помощью которого пользователь получает статистику TXT-файла без ручного подсчёта.

- считать количество символов, строк, слов и уникальных слов;
- показывать до десяти наиболее частых слов;
- поддержать чтение UTF-8, UTF-8 с BOM и CP1251;
- добавить графический выбор файла и показателя статистики;
- печатать результат в консоль и сохранять выбранный отчёт в TXT;
- сообщать пользователю об ошибках чтения или сохранения.

Технологии и инструменты

Использованы Python 3, tkinter для графического интерфейса, re для поиска слов, collections.Counter для подсчёта частот и pathlib для работы с путями. Сторонние пакеты не используются. В качестве ИИ-инструмента применялся ChatGPT: он помог подготовить и уточнить обработку кодировок, подсчёт статистики и обработку ошибок.

3 Детали реализации

Структура программы

Программа хранится в файле text_analyzer.py. Функция read_text_file пробует указанные кодировки по очереди. Функция analyze_text выделяет

слова, считает показатели и формирует отчёт. Класс TextAnalyzerApp создаёт окно, обрабатывает действия пользователя и связывает интерфейс с функциями анализа.

Функция или метод	Назначение
read_text_file	Читает файл в UTF-8 с BOM, UTF-8 или CP1251.
analyze_text	Рассчитывает выбранную статистику и формирует текст отчёта.
TextAnalyzerApp.select_file	Открывает диалог выбора файла и загружает текст.
TextAnalyzerApp.show_statistics	Показывает и печатает текущую статистику.
TextAnalyzerApp.save_report	Сохраняет отчёт в выбранный TXT-файл.
main	Создаёт окно приложения и запускает главный цикл tkinter.

Последовательность работы

- пользователь выбирает TXT-файл;
- программа читает файл в одной из поддерживаемых кодировок;
- пользователь оставляет полную статистику или выбирает отдельный показатель;
- результат появляется в окне и в консоли;
- при необходимости пользователь сохраняет результат в TXT-файл.

Код проекта:

```
import re
import tkinter as tk
from collections import Counter
from pathlib import Path
from tkinter import filedialog, messagebox


ENCODINGS = ("utf-8-sig", "utf-8", "cp1251")
STATISTIC_TYPES = (
    "Полная статистика",
    "Символы",
    "Строки",
    "Слова",
    "Частые слова",
```

```
)


def read_text_file(file_path: Path) -> str:
    for encoding in ENCODINGS:
        try:
            with open(file_path, "r", encoding=encoding) as file:
                return file.read()
        except UnicodeDecodeError:
            continue

    raise UnicodeError("Не удалось определить кодировку файла.")


def analyze_text(text: str, statistic_type: str) -> str:
    words = re.findall(r"[а-яёa-z0-9]+", text.lower())
    frequent_words = Counter(words).most_common(10)

    if statistic_type == "Символы":
        return f"Количество символов: {len(text)}"

    if statistic_type == "Строки":
        return f"Количество строк: {len(text.splitlines())}"

    if statistic_type == "Слова":
        return (
            f"Количество слов: {len(words)}\n"
            f"Количество уникальных слов: {len(set(words))}"
        )

    report_lines = []
    if statistic_type == "Полная статистика":
        report_lines.extend(
            [
                f"Количество символов: {len(text)}",
                f"Количество строк: {len(text.splitlines())}",
                f"Количество слов: {len(words)}",
                f"Количество уникальных слов: {len(set(words))}",
            ]
        )

    if statistic_type in ("Полная статистика", "Частые слова"):
        report_lines.extend(["", "Самые частые слова:"])
        if frequent_words:
            for word, count in frequent_words:
                report_lines.append(f"{word}: {count}")
        else:
            report_lines.append("В файле нет слов.")

    return "\n".join(report_lines)
```

```python
class TextAnalyzerApp:

    def __init__(self, window: tk.Tk) -> None:
        self.window = window
        self.window.title("Text File Statistics Analyzer")
        self.window.geometry("620x480")

        self.file_path: Path | None = None
        self.text = ""
        self.report = ""

        tk.Label(
            window,
            text="Text File Statistics Analyzer",
            font=("Arial", 14, "bold"),
        ).pack(pady=12)

        tk.Button(window, text="Выбрать файл", command=self.select_file).pack()

        self.file_name_label = tk.Label(window, text="Файл не выбран")
        self.file_name_label.pack(pady=8)

        self.statistic_type = tk.StringVar(value=STATISTIC_TYPES[0])
        tk.OptionMenu(
            window,
            self.statistic_type,
            *STATISTIC_TYPES,
            command=self.show_statistics,
        ).pack()

        self.result_box = tk.Text(window, height=17, width=70)
        self.result_box.pack(padx=12, pady=8, fill="both", expand=True)

        tk.Button(
            window,
            text="Сохранить отчёт",
            command=self.save_report,
        ).pack(pady=10)

    def select_file(self) -> None:
        selected_path = filedialog.askopenfilename(
            title="Выберите текстовый файл",
            filetypes=[("Текстовые файлы", "*.txt"), ("Все файлы", "*.*")],
        )
        if not selected_path:
            return

        try:
            self.text = read_text_file(Path(selected_path))
        except (OSError, UnicodeError) as error:
            messagebox.showerror("Ошибка чтения", str(error))
```

```
            return

        self.file_path = Path(selected_path)
        self.file_name_label.config(text=self.file_path.name)
        self.show_statistics()

    def show_statistics(self, selected_type: str | None = None) -> None:
        if self.file_path is None:
            return

        self.report = analyze_text(self.text, self.statistic_type.get())
        print(self.report)
        self.result_box.delete("1.0", tk.END)
        self.result_box.insert(tk.END, self.report)

    def save_report(self) -> None:
        if not self.report or self.file_path is None:
            messagebox.showinfo("Нет отчёта", "Сначала выберите текстовый файл.")
            return

        default_name = f"{self.file_path.stem}_статистика.txt"
        save_path = filedialog.asksaveasfilename(
            title="Сохранить отчёт",
            initialfile=default_name,
            defaultextension=".txt",
            filetypes=[("Текстовые файлы", "*.txt")],
        )
        if not save_path:
            return

        try:
            with open(save_path, "w", encoding="utf-8") as file:
                file.write(self.report)
        except OSError as error:
            messagebox.showerror("Ошибка сохранения", str(error))
            return

        messagebox.showinfo("Готово", "Отчёт сохранён.")


def main() -> None:
    window = tk.Tk()
    TextAnalyzerApp(window)
    window.mainloop()


if __name__ == "__main__":
    main()
```

Рисунок 1 — Запуск программы

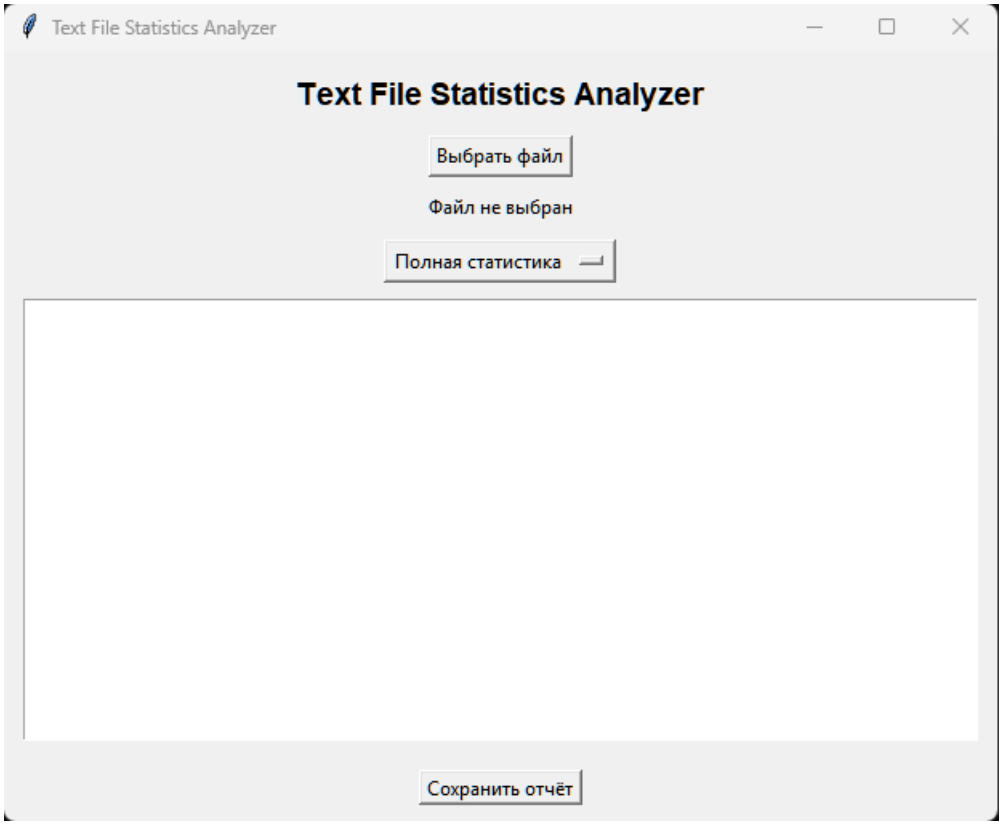


Рисунок 2 — Результат полной статистики

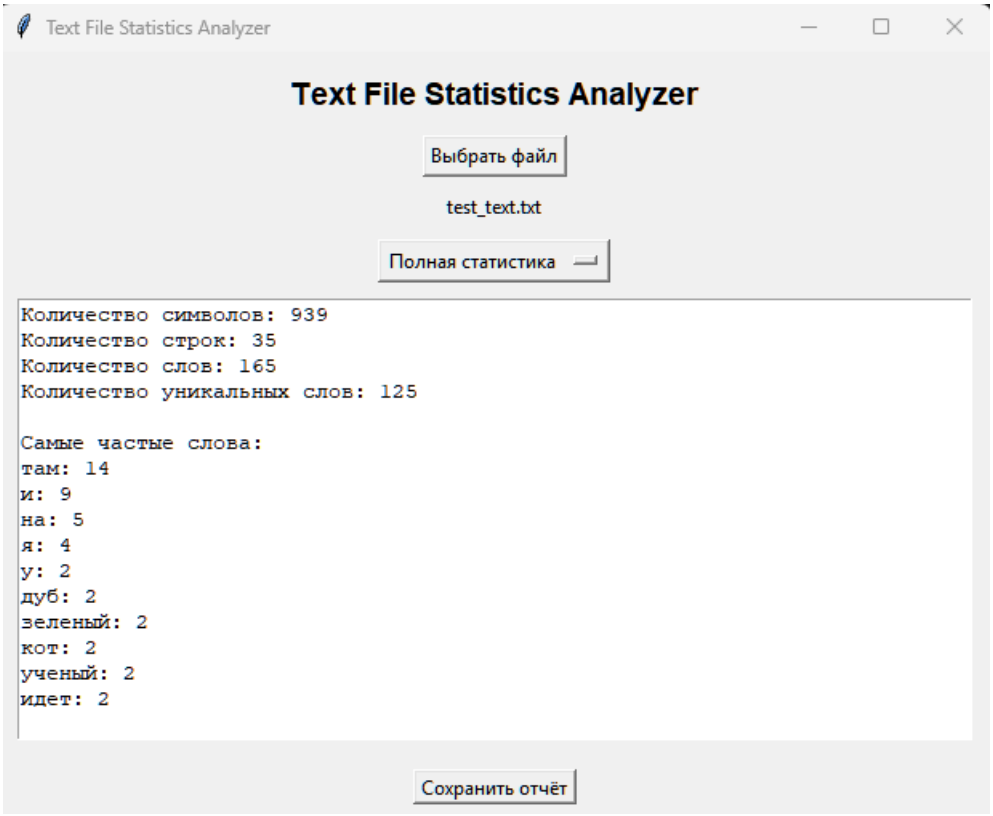


Рисунок 3 — Результат полной статистики в консоли

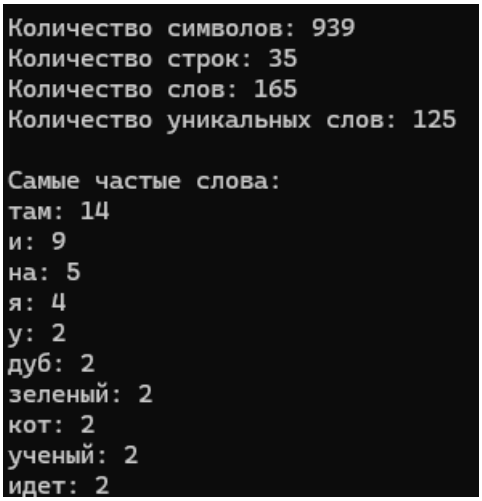
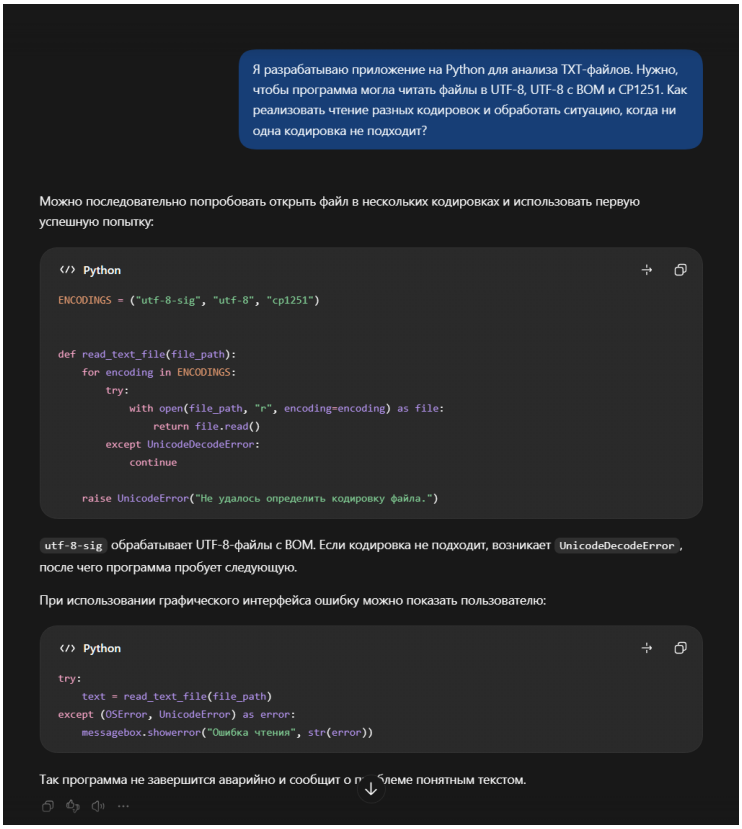


Рисунок 4 — Использование ИИ-инструмента



4 Аналитическая часть

Выбор алгоритмов и принятые решения

Перед анализом текст приводится к нижнему регистру. Регулярное выражение выделяет последовательности русских и английских букв и цифр; знаки пунктуации и пробелы служат разделителями. Counter подсчитывает частоты, а most_common(10) возвращает не более десяти наиболее частых

слов. Такой алгоритм короткий, использует стандартную библиотеку и соответствует русско- и англоязычным текстам.

Длина считается для прочитанной строки, включая пробелы и символы перевода строки после стандартной обработки Python. Количество строк определяется методом splitlines(). Уникальные слова считаются без различия регистра. Пользователь может вывести полный набор показателей или только выбранную категорию.

Контрольные примеры

Для строки «Привет мир\nПривет» ожидаются 17 символов, 2 строки, 3 слова, 2 уникальных слова и частота слова «привет» равная 2. Для пустого текста ожидаются нулевые значения и сообщение об отсутствии слов. Чтение контрольных файлов UTF-8 с BOM и CP1251 проверено; текст декодируется без добавления BOM к содержимому.

Проверочный пример	Ожидаемый результат	Полученный результат
«Привет мир\nПривет»	17 символов; 2 строки; 3 слова; 2 уникальных; «привет» — 2	Совпадает
Пустой текст	Нулевые счётчики; сообщение об отсутствии слов	Совпадает
UTF-8 с BOM и CP1251	Кириллический текст читается	Оба примера прочитаны

Интерпретация и ограничения

Программа пригодна для быстрой базовой оценки небольших и средних TXT-файлов. Поддерживаются только три указанные кодировки; другие форматы потребуют расширения списка. Подсчёт слов ориентирован на русский и английский алфавиты, поэтому слова на других языках могут учитываться не полностью. Результат можно уточнить в будущем: добавить поддержку других языков и кодировок, обработку больших файлов по частям и экспорт в CSV.

5 Инструкция по запуску

Требования к окружению

Нужен Python версии 3.10 или новее и доступный модуль tkinter. В Windows tkinter обычно устанавливается вместе с официальным дистрибутивом Python. Дополнительные пакеты через pip устанавливать не нужно.

Запуск в Windows

- Сохраните text_analyzer.py в отдельную папку.
- Откройте терминал в этой папке.
- При необходимости проверьте tkinter командой: python -m tkinter. Должно появиться демонстрационное окно.
- Запустите программу командой: python text_analyzer.py. Если используется лаунчер Windows, можно выполнить: py text_analyzer.py.
- В окне нажмите «Выбрать файл», выберите TXT, выберите вид статистики и нажмите «Сохранить отчёт», если нужно сохранить результат.

Зависимости

Внешние зависимости отсутствуют. Импортируемые модули tkinter, re, collections и pathlib входят в стандартную библиотеку Python. Если команда python не распознаётся, установите Python 3.10+ и включите добавление Python в PATH.

6 Заключение

В рамках кейса создана программа с графическим интерфейсом, которая читает TXT-файлы в нескольких распространённых кодировках, считает базовую статистику и сохраняет результат. Подготовленные контрольные примеры подтверждают работу подсчёта на русском тексте, пустом содержимом и разных кодировках.

Поставленные задачи решены средствами стандартной библиотеки Python. Дальнейшее развитие проекта может включать поддержку других языков и кодировок, настройку числа отображаемых частых слов и сохранение статистики в CSV.

7 Список использованных источников

1. Варианты кейсов для итогового проекта. Кейс 5 «Программа для подсчёта статистики текстового файла», с. 9; требования к структуре итогового проекта, с. 3. Материал предоставлен вместе с заданием.
2. Python Software Foundation. Tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html (дата обращения: 05.11.2026).
3. Python Software Foundation. Regular expression operations — re. https://docs.python.org/3/library/re.html (дата обращения: 05.11.2026).
4. Python Software Foundation. Container datatypes — collections.Counter. https://docs.python.org/3/library/collections.html#collections.Counter (дата обращения: 05.11.2026).
5. Python Software Foundation. Object-oriented filesystem paths — pathlib. https://docs.python.org/3/library/pathlib.html (дата обращения: 05.11.2026).
6. OpenAI ChatGPT. Помощь при разработке и уточнении кода приложения в рамках текущего диалога (дата обращения: 05.11.2026).